CONTROLE DE FLUXO · ENCONTRO 4 · LIGA PROTOFPGA

Folha de atividade

Nomes da dupla: _____ *Data:* __ / ___

Parte A — Atividade em aula (em duplas)

Trabalhem na mesma máquina, com o arquivo `aula4_laco.asm` aberto no MARS, valores em decimal (*Settings → Values displayed in hexadecimal* desmarcado). Cada parte tem tempo curto. Se travarem, chamem.

A1 — Observar (7 min)

1. Cliquem em Assemble. Na janela Text Segment, achem a linha de `j laco`. O que aparece na coluna Basic no lugar de `laco`? Esse endereço é de qual instrução?

---

2. Abram *Settings → Show Labels Window*. Anotem o endereço de cada rótulo. Algum deles aparece como instrução na janela Text Segment?

`laco` = _____ `pula` = _____ `fim` = _____

3. Na coluna Basic, `beq $t3, $zero, fim` aparece com um número no final. Qual? O que vocês acham que ele significa? Dica: contem instruções a partir da seguinte até chegar em `fim`.

A2 — Prever (10 min)

4. Antes de executar, escrevam:

`lw $t4 executa ___ vezes` `o slt do teste executa ___ vezes`

`Run I/O:  Soma = _____` `Maior = _____`

5. Na coluna **Bkpt** da janela Text Segment, marquem a caixa da linha `pula: addi $t0, $t0, 1`. Cliquem em Run (F5): o programa para antes dessa linha, uma vez por volta. A cada parada, anotem os valores e cliquem Run de novo.

Volta	`$t0` (i)	`$t4` (vetor[i])	`$t1` (soma)	`$t2` (maior)
1				
2				
3				
4				
5				
6				
7				
8				

6. Erraram alguma previsão? Qual, e por quê?

A3 — Modificar (7 min)

7. Desmarquem o ponto de parada. Façam o programa encontrar o **menor** elemento, trocando apenas a ordem dos dois registradores de um `slt`. Qual `slt`? Escrevam a previsão do Run I/O, montem e confiram.

8. Desfaçam a troca. Outra dupla resolveu a mesma questão de outro jeito: em vez de mexer no `slt`, trocou o `beq $t3, $zero, pula` por `bne $t3, $zero, pula`. Prevejam o que o programa imprime, testem e expliquem.

A4 — Provocar e explicar (6 min)

9. Desfaçam as trocas. Mudem `li $s1, 8` para `li $s1, 9` — uma volta a mais. Prevejam, montem e executem. O que apareceu no Run I/O?

10. De onde veio esse número? Dica: na janela Data Segment, olhem a palavra que vem logo depois do último elemento do vetor e marquem a caixa ASCII.

11. Escrevam UMA frase respondendo: quem decide quando um laço acaba — o processador ou o programa? A frase precisa ser compreensível para alguém que não esteve nesta aula.

Parte B — Desafio da semana

Enunciado

Escreva no MARS um programa que leia um número do teclado (o **limite**) e imprima quantos elementos de `vetor: .word 7, 3, 9, 2, 5, 8, 1, 6` são **maiores** que esse limite.

Use um laço `for` e um `if` dentro dele. Teste com 5 (devem ser 4), com 0 (devem ser 8) e com 9 (devem ser 0).

Pergunta extra: o que acontece se o limite for −1? Por quê?

Espaço para o seu programa:

Parte C — Exercícios de revisão

Onze perguntas, para fazer em casa. As duas últimas pedem o MARS. Tentem responder tudo antes de abrir o gabarito — conferir cedo demais desperdiça o exercício.

1. O que é um rótulo? Em que ele se transforma na montagem?

2. Qual a diferença entre `j` e `beq`? Qual dos dois, sozinho, produz um laço infinito?

3. Qual o valor de `$t0` depois de `slt $t0, $t1, $t2` em cada caso? (a) `$t1` = 5, `$t2` = 3; (b) `$t1` = 3, `$t2` = 5; (c) `$t1` = 4, `$t2` = 4; (d) `$t1` = −7, `$t2` = 2.

4. Escreva, com `slt` e `beq` ou `bne`, as duas linhas que fazem: (a) vá para `L` se `$s0` ≥ `$s1`; (b) vá para `L` se `$s0` > `$s1`. Use `$t0` como registrador auxiliar.

5. Explique por que, ao traduzir um `if`, a condição do desvio em assembly aparece invertida em relação ao C.

6. Traduza para assembly, com `a` em `$s0`, `b` em `$s1` e `c` em `$s2`:

```
if (a == b)
    c = a + b;
else
    c = a - b;
```

7. Em `for (i = 0; i < 10; i++) { … }`, quantas vezes executa o teste? E o corpo? E o incremento?

8. A instrução `0x11600008` está no endereço `0x0040001C`. Decomponha-a nos campos do formato I, diga que instrução é e calcule o endereço para onde ela desvia. Use a folha de consulta.

9. No programa da aula, cada volta soma 1 a `$t0` e 4 a `$s0`. Seria possível controlar o laço usando só `$s0`, sem o `i`? Como?

10. Sem usar o MARS, diga o que o programa abaixo imprime e quantas vezes a instrução `add` executa:

```
      li   $t0, 1
      li   $t5, 100
laco: slt  $t2, $t0, $t5
      beq  $t2, $zero, fim
      add  $t0, $t0, $t0
      j    laco
fim:  li   $v0, 1
      move $a0, $t0
      syscall
```

11. Escreva no MARS um programa que leia N do teclado e imprima `Soma =` seguido de 1 + 2 + … + N. Teste com N = 10 (55) e N = 100 (5050). O que o seu programa imprime com N = 0? Por que esse caso funciona sem nenhum código especial?

CONTROLE DE FLUXO · ENCONTRO 4 · LIGA PROTOFPGA

Folha de consulta

Tudo o que a atividade e os exercícios pedem. Rosa é desvio; âmbar, endereço; verde, valor.

Desvios e comparação

Instrução	Formato	Faz
`j L`	J · op 2	PC ← endereço de L, sempre
`beq rs, rt, L`	I · op 4	se rs = rt, PC ← L; senão, segue
`bne rs, rt, L`	I · op 5	se rs ≠ rt, PC ← L; senão, segue
`slt rd, rs, rt`	R · funct 42	rd ← 1 se rs < rt (com sinal); senão, rd ← 0

Toda comparação em duas linhas

Vá para L se…	Escreva	Pseudo do MARS
`a < b`	`slt $t, a, b  bne $t, $zero, L`	`blt a, b, L`
`a ≥ b`	`slt $t, a, b  beq $t, $zero, L`	`bge a, b, L`
`a > b`	`slt $t, b, a  bne $t, $zero, L`	`bgt a, b, L`
`a ≤ b`	`slt $t, b, a  beq $t, $zero, L`	`ble a, b, L`
`a = b` / `a ≠ b`	`beq a, b, L` / `bne a, b, L`	—

Os moldes

if

```
  slt  $t0, a, b
  beq  $t0, $zero, fim
  … então …
fim:
```

if / else

```
  slt  $t0, a, b
  beq  $t0, $zero, senao
  … então …
  j    fimse
senao: … senão …
fimse:
```

while / for

```
     li   $t0, 0
laco: slt  $t3, $t0, $s1
     beq  $t3, $zero, fim
     … corpo …
     addi $t0, $t0, 1
     j    laco
fim:
```

No `if`, o desvio usa a condição **invertida**: ele pula o bloco quando a condição do C é falsa. Cores do `for`: inicialização em azul, teste em laranja, corpo em verde, incremento em roxo.

Desvio por dentro

op	rs	rt	imediato
`beq 4 · bne 5`	`rs`	`rt`	`instruções a pular`
			destino = (PC + 4) + imediato × 4

op	endereço (26 bits)
`j 2`	`destino ÷ 4`
	os 4 bits de cima vêm do PC

Chamadas de sistema

`$v0`	Serviço	`$a0` / resultado
1	imprimir inteiro	`$a0` = `valor`
4	imprimir texto	`$a0` = `endereço` do texto
5	ler inteiro	resultado chega em `$v0`
10	encerrar	—
11	imprimir caractere	`$a0` = código (10 = nova linha)

Registradores

Nome	Nº	Binário	Nome	Nº	Binário	Nome	Nº	Binário	Nome	Nº	Binário
`$zero`	0	`00000`	`$t0`	8	`01000`	`$s0`	16	`10000`	`$t8`	24	`11000`
`$at`	1	`00001`	`$t1`	9	`01001`	`$s1`	17	`10001`	`$t9`	25	`11001`
`$v0`	2	`00010`	`$t2`	10	`01010`	`$s2`	18	`10010`	`$k0`	26	`11010`
`$v1`	3	`00011`	`$t3`	11	`01011`	`$s3`	19	`10011`	`$k1`	27	`11011`
`$a0`	4	`00100`	`$t4`	12	`01100`	`$s4`	20	`10100`	`$gp`	28	`11100`
`$a1`	5	`00101`	`$t5`	13	`01101`	`$s5`	21	`10101`	`$sp`	29	`11101`
`$a2`	6	`00110`	`$t6`	14	`01110`	`$s6`	22	`10110`	`$fp`	30	`11110`
`$a3`	7	`00111`	`$t7`	15	`01111`	`$s7`	23	`10111`	`$ra`	31	`11111`

Hexadecimal ↔ binário

0 0000	**1** 0001	**2** 0010	**3** 0011	**4** 0100	**5** 0101	**6** 0110	**7** 0111
8 1000	**9** 1001	**A** 1010	**B** 1011	**C** 1100	**D** 1101	**E** 1110	**F** 1111